

Fault Diagnosis: Decision Logic and Data Provenance

GPU Cluster Simulator · gcsim · transcribed from src/gcsim/metrics.py at commit 7bd44ef

The simulator's entire fault diagnosis is one function — `diagnose()`, `src/gcsim/metrics.py:118–303`. It is a strict `if / return` chain: six gates, evaluated in a fixed order, plus one unconditional fallback. There is no scoring, no weighting, no model. The first gate whose condition holds returns, and nothing below it ever runs.

It is called exactly once per run, from `scenarios.py:51`, and its result is merged into the run record as `summary.json["diagnosis"]`. It reads **telemetry only** — it never sees the scenario's `fault:` ground-truth label, which is what makes the verdict scoreable against that label afterwards.

1. Output schema — what a verdict actually is

The classifier returns a `Diagnosis` dataclass (`metrics.py:100–115`). Three fields carry the answer, and it is worth being precise about them because the natural way to draw this diagram — one named fault per leaf — does not match what the code emits.

Field	Domain	Notes
verdict	"NOMINAL" "HARDWARE_FAULT" "WORKLOAD_CHANGE"	Only three values. Four of the six gates return the same one.
tier	"none" "gpu" "rank" "rack" "workload"	Where the fault sits. This, not <code>verdict</code> , is what distinguishes a thermal fault from a GPU fault.
confidence	"high" "medium"	"low" is declared in the dataclass comment at <code>metrics.py:104</code> but no branch ever emits it .
slowdown_pct	float	Always <code>slowdown × 100</code> , on every branch — including <code>NOMINAL</code> . Reported as a consequence, not used as a trigger except in gates D and E.
localisation	{"racks": [...]} {"gpus": [...]} {"domains": [...]} {}	Which physical parts to look at. Empty for <code>NOMINAL</code> and <code>WORKLOAD_CHANGE</code> .
evidence, discriminators	list[str]	Human-readable justification and channel names for display. See §6 — <code>discriminators</code> overstates what the code reads.

Consequence for the diagram. A "NETWORK FAULT" leaf does not exist. A fabric fault returns `HARDWARE_FAULT / rack / high` — **the same verdict and the same tier as a thermal throttle**. The two are told apart only by `localisation` (`{"domains": [...]}` vs `{"racks": [...]}`) and by the evidence strings. Likewise there is no "RANK-LEVEL FAULT" label; that gate returns `HARDWARE_FAULT / rank / medium`.

2. The preamble — computed before any gate runs

Every "before vs. after" test in the classifier depends on one window split, defined once at `metrics.py:34–36` and applied by `_split()` at `metrics.py:57–63`:

```

BASELINE_FRACTION = 0.15      head = first 15% of rows      "before"
COMPARE_FRACTION  = 0.30      tail = last 30% of rows      "after"

def _split(df, column):                # `column` is accepted and never used
    n = len(df)
    if n < 10:
        return df, df                  # degenerate: before == after == whole frame
    head = df.iloc[: max(1, int(n * 0.15))]
    tail = df.iloc[int(n * 0.70) :]
    return head, tail

```

Two scalars are then derived from `job_performance` (`metrics.py:119-128`):

```

slowdown      = (mean(tail.iteration_time_s) - mean(head.iteration_time_s))
                / mean(head.iteration_time_s)

spread_widened = mean(tail.rank_spread_s) > max(mean(head.rank_spread_s) * 2.0,
                                                mean(head.rank_spread_s) + 1e-4)

```

`spread_widened` is computed here but consulted in **gate D only**. The window is self-baselining — there is no external healthy reference — which is precisely why gate D also carries a second, baseline-free trigger (see §4-D).

Ordering: hardware channels run before the impact gate, deliberately

The sketch's shape implies the slowdown test comes early. It does not — it is fifth. The reason is stated in the code at `metrics.py:137-141`:

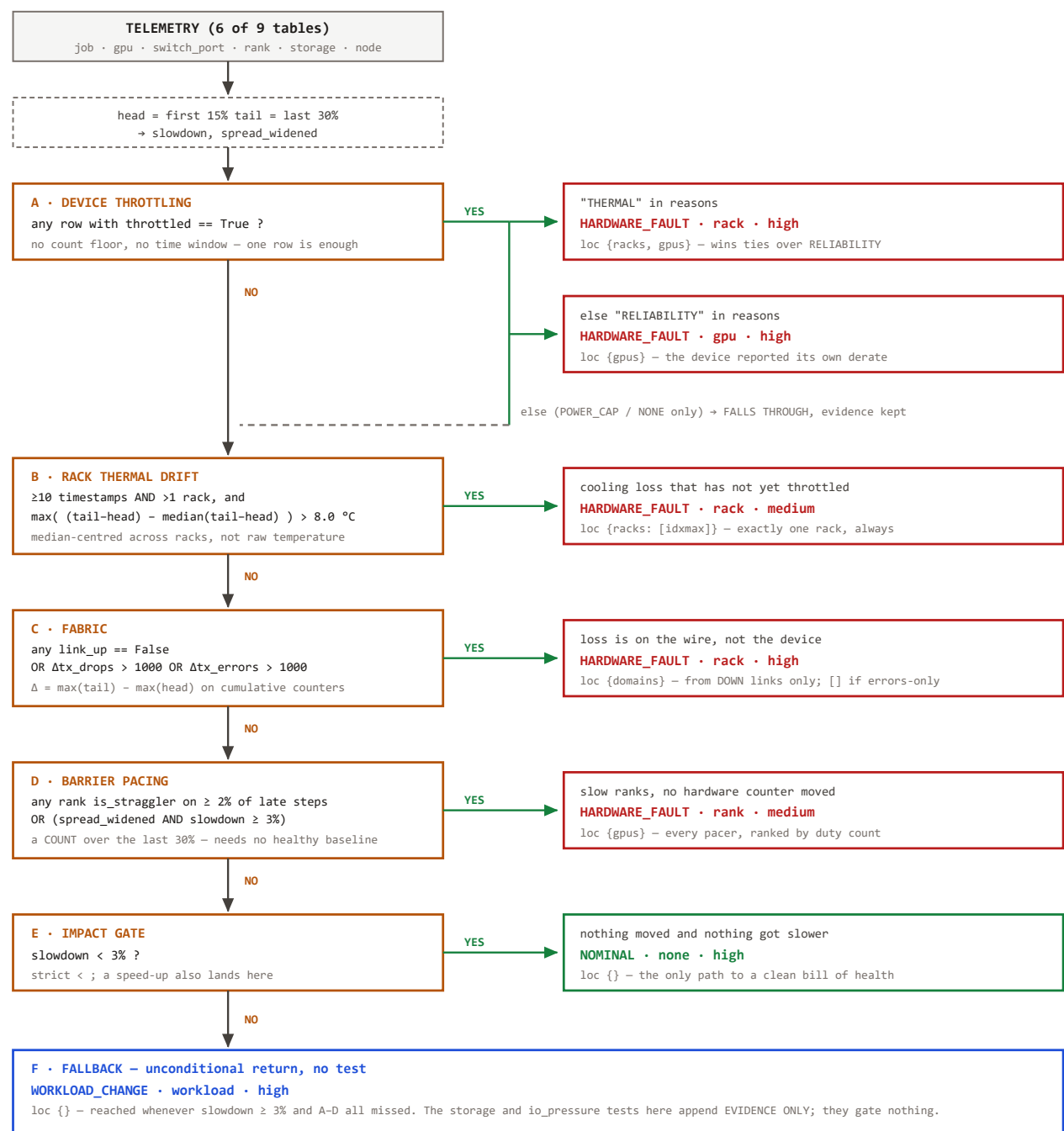
```

# A degraded link on a compute-bound job may cost almost no wall time while
# its counters scream; reporting that as "nominal" because throughput held
# up is how a fleet ends up running on a fabric that is quietly failing.

```

This is not academic. In the committed runs, straggler is diagnosed correctly at gate D with a job impact of only **+1.3%** — below the 3% threshold. Had the impact gate run first, all three straggler runs would have been called **NOMINAL**.

3. The decision tree, as the code actually evaluates it



Evaluation is strictly top-to-bottom. The first gate that returns ends the function – nothing below it is ever computed.

Source: `src/gcsim/metrics.py:118-303`. Gate letters are this document's labels; the code has no such names.

Dashed grey = fall-through (condition matched but no sub-branch did). Green = condition true. Amber = condition false.

Figure 1 — `diagnose()` in true evaluation order. Compare against the sketched version: the throttling branch can fall through, the terminals are `verdict · tier · confidence` triples rather than named faults, and fabric faults share the `rack` tier with thermal ones.

4. Gate by gate

A · Device throttling

metrics.py:143-163

```
throttled = gpu[gpu["throttled"]]
if len(throttled):
    reasons = sorted(set(throttled["throttle_reason"]))
    if "THERMAL" in reasons:      return HARDWARE_FAULT / rack / high
    if "RELIABILITY" in reasons: return HARDWARE_FAULT / gpu / high
    # no else – execution continues to gate B
```

- **Trigger:** membership in the *set* of reasons seen anywhere in the run — not a majority, not a duration, not a count. A single throttled sample at any timestamp fires the gate.
- **Tie-break:** if both reasons are present, *THERMAL* wins and the tier is *rack*.
- **Localisation caveat:** *racks* and *gpus* are collected over *all* throttled rows, not only the rows carrying the winning reason.
- **Fall-through:** if the only reason present is *POWER_CAP*, neither *if* matches and control reaches gate B — but evidence and discriminators have already been appended to, so a later verdict carries throttle evidence lines it did not act on. This path is not exercised by any committed run.

B · Rack thermal drift, before any throttle

metrics.py:165-190

```
rack_temp = gpu.pivot_table(index="timestamp", columns="rack_id",
                             values="temperature_c", aggfunc="mean")
if len(rack_temp) >= 10 and rack_temp.shape[1] > 1:
    drift = (tail - head) - (tail - head).median()      # median-centred across racks
    if float(drift.max()) > 8.0:                          # RACK_THERMAL_DRIFT_C
        return HARDWARE_FAULT / rack / medium, loc {"racks": [drift.idxmax()]}
```

- **Why it exists:** a cooling failure under a light workload may never cross the 84 °C throttle point, because the rack draws less power. Waiting for a throttle bit would miss it entirely. What is unmissable is one rack drifting away from its peers, since inlet temperature is a rack property and all 32 GPUs move together.
- **The tested quantity is a difference of differences** — each rack's (tail-mean – head-mean), then median-centred across racks. A cluster-wide warm-up cancels out; only a rack diverging from its peers survives.
- **Guards matter:** with one rack, *shape[1] > 1* is false and this gate is unreachable. With fewer than 10 telemetry samples it is skipped.
- **Localisation is always exactly one rack** (*idxmax()*), even if two racks share a chiller.

```

down          = ports[~ports["link_up"]]
drops_delta   = max(tail.tx_drops) - max(head.tx_drops)
errors_delta  = max(tail.tx_errors) - max(head.tx_errors)

if len(down) or drops_delta > 1000.0 or errors_delta > 1000.0:
    domains = sorted(down["domain_id"].dropna().unique()) # DOWN LINKS ONLY
    return HARDWARE_FAULT / rack / high, loc {"domains": domains}

```

- **Three independent triggers, OR'd.** Any one fires; both numeric tests are strict $>$ against `LINK_ERROR_FLOOR = 1000.0`.
- **Why a floor at all:** every link corrupts the odd frame at its background bit error rate (ber $1e-13$, `configs/cluster.yaml`), so a healthy fabric still trickles errors. A detector without a floor would report a fabric fault on every run.
- **Deltas are max-of-window minus max-of-window**, correct for the monotonic cumulative counters these are, but note it is *not* a per-port delta — the max may come from different ports in each window.
- **Localisation gap:** `domains` is derived only from `down` links. If the trigger was rising drops or errors while every link stayed up, `domains` is `[]` and the evidence string renders the domain as `'unknown'`.
- **Tier is rack**, because a leaf switch serves exactly one rack in this topology — the rack is simultaneously the thermal and the network failure domain.

D · Barrier pacing (stragglers)

```

late      = rank[rank["iteration"] >= rank["iteration"].max() * 0.70] # last 30%
n_steps   = late["iteration"].nunique()
paced     = late.groupby("rank_id")["is_straggler"].sum()
paced     = paced[paced >= n_steps * 0.02] # STRAGGLER_DUTY_FLOOR, inclusive >=

if len(paced) or (spread_widened and slowdown >= 0.03):
    culprits = list(paced.index) or [by_rank.idxmax()] # fallback for the spread path
    return HARDWARE_FAULT / rank / medium, loc {"gpus": [...]}

```

- **Two OR'd triggers.** The first — a duty-cycle count — needs no baseline at all. The second is the baseline-dependent one, and requires *both* a widened spread and $\geq 3\%$ slowdown.
- **Why a count and not a mean:** for an intermittent fault, a rank derated for a tenth of the window barely lifts its own average above ordinary silicon variation, while the mean happily promotes whichever healthy rank is the fleet's slowest. The count also finds every member of a cohort, where an `argmax` can only ever name one.
- **Why the floor sits at 2%:** silicon variation alone puts the unluckiest rank of a healthy fleet near 0.5% of timesteps; a genuinely derated rank sits an order of magnitude above that. The gate is wide.
- **Why this gate precedes the impact gate:** the before/after comparison silently assumes the run began healthy. The `straggler` scenario violates that by design — its episodes start at iteration 1 — so it shifts neither mean nor spread between windows and would be waved through as nominal.
- **Note the operator asymmetry:** this gate uses `slowdown >= 0.03`; gate E uses `slowdown < 0.03`. Exactly 3.0% therefore belongs to D, not E.
- **Localisation deliberately carries GPUs only**, never rank ids — the dashboard panel flattens every value it is given, and bare rank numbers beside device ids would be unreadable.

E • Impact gate

metrics.py:271-280

```
if slowdown < 0.03:                                # SLOWDOWN_THRESHOLD, strict <
    return NOMINAL / none / high
```

- The only path to `NOMINAL`. Reached only when every hardware channel is clean *and* nobody is pacing the barrier.
- A negative slowdown (the run got faster) also satisfies this and returns `NOMINAL`.

F • Fallback

metrics.py:281-303

```
lat_ratio = mean(tail.write_latency_ms) / mean(head.write_latency_ms)
io_delta  = mean(tail.io_pressure) - mean(head.io_pressure)

if lat_ratio > 1.5: evidence.append(...)    # EVIDENCE ONLY
if io_delta  > 0.02: evidence.append(...)  # EVIDENCE ONLY

return WORKLOAD_CHANGE / workload / high  # unconditional
```

The storage and I/O tests do not decide anything. They only append evidence strings. `WORKLOAD_CHANGE` is returned whenever the run slowed by $\geq 3\%$ and gates A–D all missed — even with completely flat storage latency and I/O pressure. Drawing this leaf as "WORKLOAD / STORAGE CHANGE" behind a storage test, as the sketch does, implies a branch that is not there.

This is the classifier's residual bucket: it is where an unexplained slowdown goes. It is labelled `WORKLOAD_CHANGE` because a workload phase change is the most common benign cause, not because the code established one.

5. Data provenance — where every input comes from

The classifier reads **19 columns across 6 tables**. Every one is produced by the simulator itself and written to `runs/<run_id>/*.parquet`; the column set is a hard contract enforced by `telemetry.py`'s `SCHEMAS` (lines 21–68) and `conform()`, which fails on any missing or extra column.

Column	Table	Physically produced by	Sampled / recorded at	Used by
<code>iteration_time_s</code>	<code>job_performance</code>	Simulated wall time of one timestep: <code>compute + halo + barrier + allreduce + checkpoint</code>	<code>simulator.py</code> <code>_record_performance</code>	preamble, E
<code>rank_spread_s</code>	<code>job_performance</code>	<code>busy.max()</code> – <code>busy.min()</code> where <code>busy = compute + halo</code>	<code>simulator.py:408</code>	preamble, D
<code>throttled</code>	<code>telemetry_gpu</code>	<code>clock_factor < 1 – 1e-12</code> after the three governors are minimised	<code>thermal.py:173</code> → <code>samplers.py:214</code>	A
<code>throttle_reason</code>	<code>telemetry_gpu</code>	<code>argmin</code> over [thermal, power, reliability] clock factors; ties resolve to lowest index, so THERMAL > POWER_CAP > RELIABILITY	<code>thermal.py:171</code> → <code>samplers.py:215</code>	A
<code>temperature_c</code>	<code>telemetry_gpu</code>	First-order RC lag: <code>steady = T_inlet + P·R_th</code> , then <code>T ← steady + (T – steady)·e^{-(dt/τ)}</code> with <code>R_th = 0.06 °C/W</code> , <code>τ = 20 s</code>	<code>thermal.py:124–126</code> → <code>samplers.py:212</code>	B
<code>rack_id</code> , <code>gpu_id</code>	<code>telemetry_gpu</code>	Static topology identifiers (<code>r1n2g5</code> form)	<code>samplers.py:101–104</code>	A, B
<code>link_up</code>	<code>telemetry_switch_port</code>	<code>port.up</code> . Set <code>False</code> only by the <code>leaf_uplink_failure</code> injection (<code>faults.py:286</code>)	<code>samplers.py:300</code>	C
<code>tx_drops</code>	<code>telemetry_switch_port</code>	Cumulative. Buffer overflow from the M/M/1 queue model: <code>overflow/(overflow + buffer_packets)</code> , <code>buffer_packets = 2000</code>	<code>network.py:264–265</code> → <code>:347</code> accumulate	C
<code>tx_errors</code>	<code>telemetry_switch_port</code>	Cumulative. Per-port physical <code>error_rate</code> — background BER <code>1e-13</code> , or the injected <code>0.02</code> on surviving uplinks after a failure	<code>network.py:348</code> <code>faults.py:289</code>	C
<code>domain_id</code>	<code>telemetry_switch_port</code>	Rack id on a leaf switch; null on spine ports (a spine belongs to no single domain)	<code>samplers.py:271–313</code>	C
<code>port_id</code>	<code>telemetry_switch_port</code>	Static port identifier	<code>samplers.py</code>	C (evidence)
<code>is_straggler</code>	<code>rank_performance</code>	<code>busy > median(busy) · (1 + 0.05)</code> — per timestep, against that timestep's own fleet median	<code>simulator.py:318–319</code>	D
<code>compute_time_s</code>	<code>rank_performance</code>	<code>Cells × seconds_per_cell_update</code> , divided by the rank's effective clock factor	<code>simulator.py:387</code>	D (fallback)

halo_wait_s	rank_performance	Misnomer: this is the halo exchange <i>duration</i> , i.e. work — not a wait. The only real wait in the table is <code>allreduce_wait_s</code> . Kept as-is because renaming would invalidate every run on disk.	simulator.py:383–386	D (fallback)
rank_id, gpu_id, iteration	rank_performance	Rank index, its placed device, timestep number	simulator.py	D
write_latency_ms	telemetry_storage	M/M/1 queueing: $\text{base_write_latency_ms} / (1 - \rho)$, time-weighted over the sample window	storage.py:99, :142	F (evidence only)
io_pressure	telemetry_node	$\min(1, \text{output_fraction} + \text{background})$ — driven by checkpoint output staging	samplers.py:234	F (evidence only)
timestamp	gpu, port, storage, node	Sample tick time, on the telemetry clock	samplers.py:203	all window splits

The causal chain behind the thermal channel

Gates A and B both ultimately read one physical loop — the only feedback path in the model:

workload phase	→ resource demand	→ sm_occupancy
occupancy + clock	→ power_w	(idle + dynamic, clamped to the 700 W board cap)
power + cooling	→ temperature_c	$dT/dt = (P \cdot R_{th} - (T - T_{inlet})) / \tau$
temperature (+3 °C hysteresis)	→ throttled, throttle_reason	
throttled	→ clock_mhz	→ effective performance → rank compute time

A cooling fault is injected upstream of all of this, as a change to `nominal_efficiency` — it raises `T_inlet` for one rack. Nothing in the fault injector can touch telemetry directly: `faults.py` may import only `topology` and `workload`, and has no path to `telemetry`, `samplers` or `metrics`. Every signal the classifier sees is therefore a genuine downstream consequence of a physical change, not a planted marker.

Two different clocks — and why it matters for the windows

Grain	Cadence	Rows per run	Tables
Telemetry sample	1.0 s	~202	<code>telemetry_gpu</code> , <code>_node</code> , <code>_nic</code> , <code>_switch_port</code> , <code>_switch_aggregate</code> , <code>_storage</code>
Simulation timestep	~40–800 ms, variable	1000	<code>rank_performance</code> , <code>job_performance</code>

The 1 Hz figure is `configs/cluster.yaml:127`, `sample_interval_s: 1.0`, chosen because real fleets poll DCGM at about 1 Hz — so stalls shorter than a second are genuinely invisible, a limitation of the model *and* of real monitoring. The consequence for the diagnosis is that `_split()`'s "first 15% / last 30%" means ~30 and ~61 rows for a GPU-table split, but 150 and 300 iterations for a job-table split. They are different windows over the same run.

Aggregation applied before the classifier ever sees a number

- **Power** — EWMA with `alpha = 0.35` (`thermal.py:120–121`). The sensor, not the physics.
- **Temperature** — RC lag, `τ = 20 s`, plus a warm-start that snaps both lags at `t≈0` so a `~3τ` cold-start transient does not run through the injection points and contaminate every before/after comparison.
- **Occupancy** — time-weighted over the sample window, not conditional on a kernel doing useful work. This is the whole point: a rank stalled at a barrier keeps a spin kernel resident, so utilisation stays pinned near 100% while occupancy falls.
- **Counters** — cumulative on the wire, differenced into rates at sample time.

- **Queue depth** — the high-water mark during the exchange, not a window mean. A link can average 4% utilisation and still queue hard.

At the diagnosis layer itself there is **no** further smoothing: plain means over head and tail, one median-centring in gate B, one count in gate D. No rolling means, no percentiles, no EWMA.

Where the thresholds live

All six detector constants are hard-coded in Python at `metrics.py:34–54` — **none of them are in YAML**. The physics thresholds they react to are in `configs/cluster.yaml`. Anyone wanting to sweep detector sensitivity has to edit the source.

Constant	Value	Defined in	Gate
<code>BASELINE_FRACTION</code>	<code>0.15</code>	<code>metrics.py:35</code>	all window splits
<code>COMPARE_FRACTION</code>	<code>0.30</code>	<code>metrics.py:36</code>	all window splits
<code>SLOWDOWN_THRESHOLD</code>	<code>0.03</code>	<code>metrics.py:39</code>	D, E
<code>SPREAD_WIDEN_FACTOR</code>	<code>2.0</code>	<code>metrics.py:41</code>	D
<code>STRAGGLER_DUTY_FLOOR</code>	<code>0.02</code>	<code>metrics.py:47</code>	D
<code>LINK_ERROR_FLOOR</code>	<code>1000.0</code>	<code>metrics.py:51</code>	C
<code>RACK_THERMAL_DRIFT_C</code>	<code>8.0</code>	<code>metrics.py:54</code>	B
<code>thermal_slowdown_c</code>	<code>84.0 °C</code>	<code>cluster.yaml:45</code>	physics — sets throttled for A
<code>thermal_hysteresis_c</code>	<code>3.0 °C</code>	<code>cluster.yaml:47</code>	physics — stops throttle flapping
<code>straggler_rel_threshold</code>	<code>0.05</code>	<code>cluster.yaml:128</code>	physics — sets <code>is_straggler</code> for D
<code>sample_interval_s</code>	<code>1.0 s</code>	<code>cluster.yaml:127</code>	telemetry cadence

6. Signals the classifier does *not* use

Worth stating explicitly, because both the sketch and the code's own `discriminators` field imply a wider read set than exists.

- **Three of the nine telemetry tables never reach `diagnose()`.** `telemetry_nic`, `telemetry_switch_aggregate` and `events` are produced, schema-validated and written to disk, but the classifier never loads them. In particular **no congestion or oversubscription signal is consulted** — gate C is link-state and error counters only.
- **`discriminators` is a display label list, not the read set.** It names `clock_mhz` and `power_w` (gate A), `oversubscription_ratio` (gate C), and `sm_occupancy_pct` and `allreduce_wait_s` (gate D) — **none of which are read by the code that names them**. Treat that field as "channels a human should go look at", not "channels that produced this verdict".
- **The dashboard's signature matrix is a separate implementation.** `dashboard/build.py:305–379` computes an up/down/flat fingerprint per channel and reaches no verdict. It shares `STRAGGLER_DUTY_FLOOR` by import, but duplicates the rack-drift `8.0` and link-error `1000` as literals (`build.py:341`, `374`) and uses a looser guard (`>= 6` timestamps vs `>= 10`). Tune a constant in `metrics.py` and the displayed signature can silently disagree with the verdict beside it.

7. Worked examples — all 18 committed runs

Six scenarios × three mesh resolutions, seed 42, read from `runs/*/summary.json`. The gate column is inferred from the recorded `tier`, confidence and localisation shape, which identify the exit branch uniquely.

Scenario	Mesh	Gate	Verdict	Tier	Conf.	Impact	Localisation	Truth
healthy	coarse	E	NOMINAL	none	high	+0.55%	{}	✓ no fault
healthy	medium	E	NOMINAL	none	high	+0.28%	{}	✓ no fault
healthy	fine	E	NOMINAL	none	high	+0.22%	{}	✓ no fault
straggler	coarse	D	HARDWARE_FAULT	rank	medium	+1.55%	gpus: r1n1g0, r2n3g6, r1n1g5	✓ rank
straggler	medium	D	HARDWARE_FAULT	rank	medium	+1.29%	gpus: r1n1g0, r2n3g6, r1n1g5	✓ rank
straggler	fine	D	HARDWARE_FAULT	rank	medium	+1.27%	gpus: r1n1g0, r2n3g6, r1n1g5	✓ rank
network_domain	coarse	C	HARDWARE_FAULT	rack	high	+17.02%	domains: r2	✓ rack
network_domain	medium	C	HARDWARE_FAULT	rack	high	+19.82%	domains: r2	✓ rack
network_domain	fine	C	HARDWARE_FAULT	rack	high	+13.47%	domains: r2	✓ rack
thermal	coarse	B	HARDWARE_FAULT	rack	medium	+0.55%	racks: r1	✓ rack
thermal	medium	A	HARDWARE_FAULT	rack	high	+8.92%	32 gpus in r1 + racks: r1	✓ rack
thermal	fine	A	HARDWARE_FAULT	rack	high	+13.82%	32 gpus in r1 + racks: r1	✓ rack
gpu_degradation	coarse	A	HARDWARE_FAULT	gpu	high	+24.18%	gpus: r3n1g2	✓ gpu
gpu_degradation	medium	A	HARDWARE_FAULT	gpu	high	+26.74%	gpus: r3n1g2	✓ gpu
gpu_degradation	fine	A	HARDWARE_FAULT	gpu	high	+27.49%	gpus: r3n1g2	✓ gpu
phase_change	coarse	F	WORKLOAD_CHANGE	workload	high	+9.79%	{}	✓ no fault
phase_change	medium	F	WORKLOAD_CHANGE	workload	high	+10.11%	{}	✓ no fault
phase_change	fine	F	WORKLOAD_CHANGE	workload	high	+10.37%	{}	✓ no fault

Scoring rule (`cli.py:109-110`): a run is correct if `is_fault` and the verdict is `HARDWARE_FAULT`, or if `not is_fault` and the verdict is anything else. It is a binary collapse — `NOMINAL` and `WORKLOAD_CHANGE` both count as "not a fault". Current score: **18/18**.

The result worth reading twice

The same physical fault exits at a different gate depending on the workload. `thermal` injects an identical cooling degradation — rack r1, efficiency 0.30, ramped over 60 iterations — in all three runs. On the **medium** and **fine** meshes the dies reach 88.7 °C and 90.8 °C, cross the 84 °C threshold, and 32 GPUs throttle for ~16–18% of samples: gate A fires with high confidence. On the **coarse** mesh the same lost cooling only reaches 55.4 °C, because the lighter workload draws less power — **nothing throttles at all** (`throttled_sample_fraction` = 0.0), and the job slows by 0.55%, which is under the impact gate. Gate A misses, gate E would have said `NOMINAL`. Only gate B catches it, on the cross-rack drift, at medium confidence.

That is the case gate B exists for, and it is the strongest argument in the whole design for checking hardware channels before job impact. Under a light workload a failed CRAC is invisible to every throttle bit and to throughput alike.

The `straggler` rows make the complementary point from the other direction: all three are diagnosed at +1.3–1.6% impact, well below the 3% gate, and all three are correct. A detector gated on impact would have missed every one.

8. Corrections against the sketched diagram

As sketched	As implemented
"Any GPU throttling? → YES" always terminates in a fault	A <code>POWER_CAP</code> -only throttle matches neither sub-branch and falls through to gate B, carrying its already-appended evidence lines with it (<code>metrics.py:146–163</code>)
Leaves named <code>RACK/COOLING FAULT</code> , <code>GPU FAULT</code> , <code>NETWORK FAULT</code> , <code>RANK-LEVEL FAULT</code>	No such labels exist. Outputs are <code>verdict + tier</code> pairs, and fabric and thermal faults share the identical <code>HARDWARE_FAULT</code> / <code>rack output</code> — separable only by localisation
"Rack temperature drift > threshold"	Two guards first (<code>≥10 timestamps</code> , <code>>1 rack</code>), and the tested quantity is the median-centred cross-rack delta, strict <code>> 8.0 °C</code> . A uniform cluster-wide warm-up cancels to zero and does not fire
"Network fault? link down / errors ↑ / drops ↑"	Correct as an OR. But the deltas are max-of-tail – max-of-head on cumulative counters against a floor of 1000, and the domain list comes only from down links — an errors-only trigger localises to <code>[]</code>
"Rank straggler?" as one test	Two OR'd triggers: a duty count <code>≥ 2%</code> of late-window steps (baseline-free), or <code>spread_widened AND slowdown ≥ 3%</code>
"Slowdown < 3%" implies impact gates the hardware branches	Position is right — it is fifth — but the implication is backwards: all four hardware gates run before it, deliberately, so a fault with no wall-clock cost is still reported
"WORKLOAD / STORAGE CHANGE" implies a storage test	There is no test. <code>lat_ratio > 1.5</code> and <code>io_delta > 0.02</code> append evidence strings only ; the verdict is returned unconditionally (<code>metrics.py:298–303</code>)
Storage / node I/O drawn as diagnostic inputs alongside GPU and network	They are read, but only to phrase evidence. Meanwhile <code>telemetry_nic</code> , <code>telemetry_switch_aggregate</code> and <code>events</code> are never read at all

9. Honest limits

- **This is a small rule set, not a detector anyone should deploy.** The repository says so itself (`README.md`). It scores 18/18 on the current six-scenario matrix, which is a matrix it was written against — that number measures internal consistency, not generalisation.
- **It cannot localise a degraded-but-up port.** Gate C reports the affected *domain*, never a port id, and only when a link is actually down.
- **Four of the six gates are self-baselining** against the first 15% of the same run. Any fault already present at t=0 is invisible to them; gate D's duty count is the only check that survives never having seen the cluster healthy.
- **WORKLOAD_CHANGE is a residual, not a finding.** It means "slowed down, and none of the five specific checks matched" — nothing more.
- **No congestion signal is consulted.** `oversubscription_ratio` and `max_queue_depth` are computed, validated, stored, listed as discriminators — and never read by the classifier.

Generated from source at commit 7bd44ef (branch main, clean working tree). Every threshold, operator and line reference in this document was read from `src/gcsim/metrics.py`, `src/gcsim/models/thermal.py`, `src/gcsim/engine/simulator.py`, `src/gcsim/samplers.py`, `src/gcsim/telemetry.py` and `configs/cluster.yaml`. Run results in \$7 were read from the 18 `runs/*/summary.json` files committed in the repository. No source file was modified.